

26 Transformer升级之路：16、“复盘”长度外推技术

Jan By 苏剑林 | 2024-01-26 | 98308位读者

回过头来看，才发现从第7篇《Transformer升级之路：7、长度外推性与局部注意力》开始，“Transformer升级之路”这个系列就跟长度外推“杠”上了，接连9篇文章（不算本文）都是围绕长度外推展开的。如今，距离第7篇文章刚好是一年多一点，在这一年间，开源社区关于长度外推的研究有了显著进展，笔者也逐渐有了一些自己的理解，比如其实这个问题远不像一开始想象那么简单，以往很多基于局部注意力的工作也不总是有效，这暗示着很多旧的分析工作并没触及问题的核心。

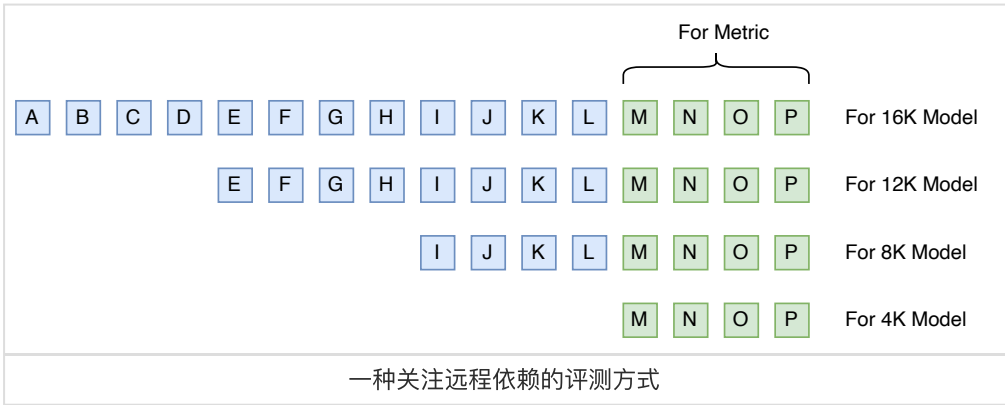
在这篇文章中，笔者尝试结合自己的发现和认识，去“复盘”一下主流的长度外推结果，并试图从中发现免训练长度外推的关键之处。

问题定义

顾名思义，免训练长度外推，就是不需要用长序列数据进行额外的训练，只用短序列语料对模型进行训练，就可以得到一个能够处理和预测长序列的模型，即“Train Short, Test Long”。那么如何判断一个模型能否用于长序列呢？最基本的指标就是模型的长序列Loss或者PPL不会爆炸，更加符合实践的评测则是输入足够长的Context，让模型去预测答案，然后跟真实答案做对比，算BLEU、ROUGE等，LongBench就是属于这类榜单。

但要注意的是，长度外推应当不以牺牲远程依赖为代价——否则考虑长度外推就没有意义了，倒不如直接截断文本——这意味着通过显式地截断远程依赖的方案都需要谨慎选择，比如ALIBI以及《Transformer升级之路：7、长度外推性与局部注意力》所列举的大部分方案，还有带显式Decay的线性RNN，这些方案当序列长度足够大时都表现为局部注意力，即便有可能实现长度外推，也会有远程依赖不足的风险，需要根据自己的场景斟酌使用。

如何判断在长度外推的同时有没有损失远程依赖呢？比较严谨的是像《Transformer升级之路：12、无限外推的ReRoPE? 》最后提出的评测方案，准备足够长的文本，但每个模型只算每个样本最后一段的指标，如下图所示：



比如，模型训练长度是4K，想要看外推到16K的效果，那么我们准备一个16K tokens的测试集，4K的模型输入每个样本最后4K tokens算指标，8K模型输入每个样本最后8K tokens但只算最后4K tokens算指标，12K模型输入每个样本最后12K tokens但只算最后4K tokens算指标；依此类推。这样一来，不同长度的模型算的都是同一段tokens的指标，不同的只是输入的Context不一样，如果远程依赖得以有效保留，那么应该能做到Context越长，指标越好。

旋转位置

谈完评测，我们回到方法上。文章开头我们提到“旧的分析工作”，这里“新”、“旧”的一个主要特点是“旧”工作多数试图自行设置新的架构或者位置编码来实现长度外推，而最近一年来的“新”工作主要是研究带旋转位置编码（RoPE）的、Decoder-Only的Transformer模型的长度外推。

先说个题外话，为什么如今大部分LLM的位置编码都选择了RoPE呢？笔者认为主要有几点原因：

- 1、RoPE不带有显式的远程衰减，这对于旨在Long Context的模型至关重要；
- 2、RoPE是一种真正的位置编码，通过不同频率的三角函数有效区分了长程和短程，达到了类似层次位置编码的效果，这也是Long Context中比较关键的一环；
- 3、RoPE直接作用于Q、K，不改变Attention的形式，与Flash Attention更契合，更容易Scale Up。

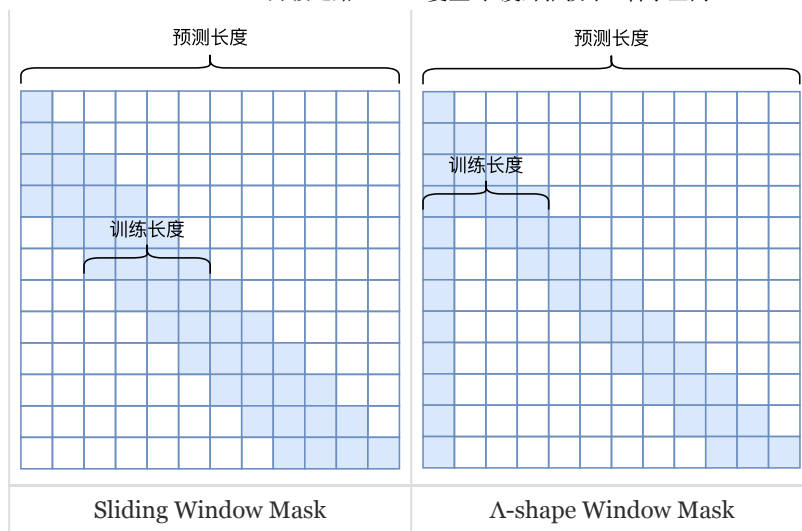
相比之下，诸如ALIBI、KERPLE等，虽然有时也称为位置编码，但它们实际上只是一种Attention Bias，没有太多位置信息，且不适用于Encoder，能用于Decoder大体上是因为Decoder本身的下三角Mask就已经有较为充分的位置Bias了，额外的Attention Bias只是锦上添花。此外它们无法在单个头内有效区分长程和短程，而是要通过在不同头设置不同的Decay因子来实现，这也意味着它们用于单头注意力（比如GAU）的效果会欠佳。

说这么多优缺点的对比，看起来像是“王婆卖瓜，自卖自夸”，其实不然，这只是为了跟大家交换一下观点，因为之前也有读者提出过相同的问题。作为RoPE的提出者，笔者对RoPE的理解不见得一定比大家深刻，毕竟当时提出RoPE的初衷纯粹是好玩，当时的想法是有效就很不错了，能媲美Learnable的绝对位置编码就是非常好的消息了。所以，既然是“意料之外”，那么“作者本人也没多透彻的认识”这件事，也是“情理之中”了。

窗口截断

好像又把话题扯偏了。简单来说，其实上两节的内容主要是想表达的观点是：目前看来，RoPE对于Long Context来说是足够的，所以研究RoPE的长度外推是有价值的，以及我们在选择长度外推方案时，不应牺牲远程依赖的能力。

在本站最早讨论长度外推的《Transformer升级之路：7、长度外推性与局部注意力》一文中，我们判断长度外推是一个预测阶段的OOD（Out Of Distribution）的问题，尽管用今天的视角看，这篇文章的一些评述已经显得有点过时，但这个根本判断是依然还算正确，放到RoPE中，就是推理阶段出现了没见过的相对距离。为此，一个看上去可行的方案是引入Sliding Window的Attention Mask，如下图左所示：



当然，由于强行截断了窗口外的注意力，所以这个方案并不满足“不牺牲远程依赖的能力”的原则，但我们可以只将它作为一个Baseline看待。很遗憾的是，即便做出了如此牺牲，这个方案却是不Work的——连最基本的PPL不爆炸都做不到！对这个现象的深入分析，先后诞生《LM-Infinite: Simple On-the-Fly Length Generalization for Large Language Models》和《Efficient Streaming Language Models with Attention Sinks》两篇论文，并给出了几乎一样的答案。但事实上，在更早的几个个月前，一位“业外人士”就发现了相同的结论，并发表在知乎专栏文章《Perpetual Sampling Technical Report》上。

答案可能让人意外：**开头的几个Token很重要，不能扔掉**。所以最后可用的Window Mask应该如上图右（LM-Infinite这篇论文管它叫“ Δ -Mask”）。

为什么开头的Token会占据如此重要的地位呢？目前有两个不同的理解角度：

- 1、开头的几个Token是绝对位置的“锚点”：顾名思义，相对位置编码原则上只能识别相对位置，但有些任务可能比较依赖绝对位置，通过开头几个绝对位置约等于0的Token作为“标的”，每个Token就能够测出自己的绝对位置，而去掉开头几个Token后则缺失了这一环，从而完全打乱了注意力模式导致PPL爆炸；
- 2、开头的几个Token是注意力的“回收站”：由于注意力求和为1，所以注意力一定会分配到某些Token上，但有些情况下模型可能会发现“没什么Token值得注意的”，这时它选择将一部分注意力放到没什么信息量的前几个Token上，起到“不注意”的作用，去掉它们后模型会强行将注意力分配到其他无关的Token，从而扰乱了注意力模式。

其实说白了，就是实测发现大部分情况下，前几个Token的注意力占比还是很重的，所以不能去掉，去掉注意力就全乱了。至于为什么很重，就看大家的想象力了。

位置内插

窗口截断的方式固然可以作为长度外推的一个不错的Baseline，同时“锚点”或者“回收站”的结果也让我们对注意力机制的工作方式有了进一步的理解，但正如前面所说，这是通过强行截断窗口外的注意力、牺牲远程依赖换来的，因此还不是最终的解决方案。

相对位置的OOD，直接表现就是预测阶段的相对位置超出了训练时的范围，由于没有被训练过，“越界”部分的行为无法预估。为此，一位网名为“kaiokendev”的网友在他的博客《<https://kaiokendev.github.io/til#extending-context-to-8k>》中提出了一个非常朴素的解决办法——“位置内插”——将预测的长文本的位置编码乘上因子 $\frac{L_{train}}{L_{test}}$ ，缩放到训练长度范围内，如下式所示（式中的位置都是相对位置）。没过多久，Meta在论文《Extending

g Context Window of Large Language Models via Positional Interpolation》中也发布了同样的方法，命名为“Positional Interpolation (PI)”，并补充了较为充分的实验结果。

训练阶段： $(1, 2, \dots, n-1, n)$

$$\text{预测阶段：}(1, 2, \dots, n, \underbrace{n+1, \dots, 4n-1, 4n}_{\text{远处越界}}) \xrightarrow{\text{内插}} \left(\underbrace{\frac{1}{4}, \frac{2}{4}, \frac{3}{4}}_{\text{局部失真}}, \dots, n - \frac{1}{4}, n \right) \quad (1)$$

然而，位置内插并不算长度外推方案，至少不是免训练的长度外推方案，因为位置内插之后同样会有PPL爆炸的问题。原因也不难理解，尽管位置内插避免了远处的位置越界问题，但这同时压缩了邻近Token的距离，严重扰乱了模型的局部分辨率，而众所周知语言模型本身就是一个非常依赖于局部关系的任务，所以扰乱了局部自然就没法预测准了。

不过，这也并非说位置内插就没有价值了。我们知道，需要长度外推的读者，无外乎是两种情况：一种是没有资源去做长文本微调，希望能够从短文本模型直接得到一个可用的长文本模型，这种需求对长度外推的效果要求会比较高，位置内插就不适合他们了；另一种是有资源去做长文本微调，研究长度外推纯粹是为了得到一个更好的初始化模型，这种情况对模型修改带来的初始损失容忍度比较高，只要能够通过微调快速弥补回损失掉的效果即可，位置内插正好是属于此类方法。Meta的论文显示，经过PI之后，仅需1000步左右的长文本训练，就可以得到一个行之有效的长文本模型，这比不做任何修改直接微调的训练效率高很多。

保近压远

直接外推的问题是远处越界，而位置内插的问题是局部失真，看上去两者是互补的，能不能集两者之长呢？这就是《Transformer升级之路：12、无限外推的ReRoPE?》所提出的Leaky ReRoPE，以及它的极限版本ReRoPE。

基于上一节的分析，我们不难推测实现免训练长度外推的要领是“保近压远”，即“保证局部不失真”和“压缩远处不越界”，Leaky ReRoPE通过一个非常直接的思路实现了这一点：它先设定一个窗口大小 w 内，将相对位置分

$$\left(\begin{array}{cccccccccccc} 0 & & & & & & & & & & & \\ 1 & 0 & & & & & & & & & & \\ 2 & 1 & 0 & & & & & & & & & \\ \ddots & 2 & 1 & 0 & & & & & & & & \\ w-1 & \ddots & 2 & 1 & 0 & & & & & & & \\ w & w-1 & \ddots & 2 & 1 & 0 & & & & & & \\ w+\frac{1}{k} & w & \ddots & \ddots & 2 & 1 & 0 & & & & & \\ w+\frac{2}{k} & w+\frac{1}{k} & \ddots & \ddots & \ddots & 2 & 1 & 0 & & & & \\ \ddots & w+\frac{2}{k} & \ddots & \ddots & \ddots & \ddots & 2 & 1 & 0 & & & \\ \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & 2 & 1 & 0 & & \\ \ddots & \ddots & \ddots & w+\frac{2}{k} & w+\frac{1}{k} & w & w-1 & \ddots & 2 & 1 & 0 & \\ w+\frac{L-1-w}{k} & \ddots & \ddots & \ddots & w+\frac{2}{k} & w+\frac{1}{k} & w & w-1 & \ddots & 2 & 1 & 0 \end{array} \right) \quad (2)$$

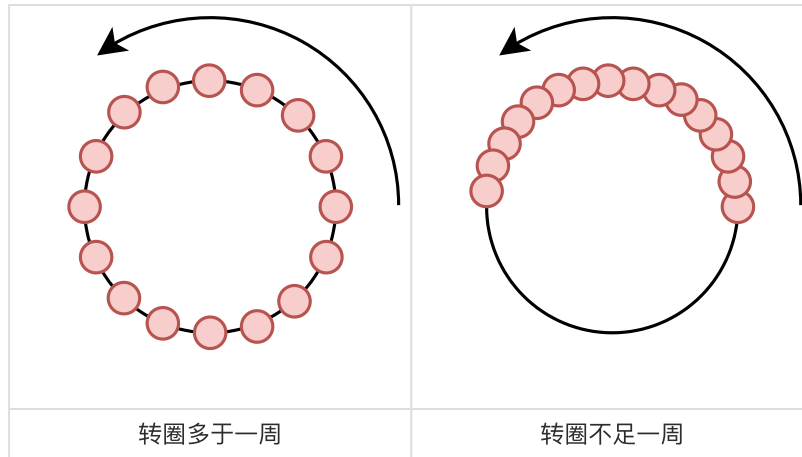
Leaky ReRoPE和ReRoPE的主要问题在于它们的代码实现稍微有点麻烦。跟Attention Bias类的位置编码不同，RoPE没法通过先构造相对位置矩阵然后才计算相对位置编码的方式来实现（那样效率太低），只能通过绝对位置编码的方式来实现相对位置编码，这意味着它只能实现线性增长的相对位置，而Leaky ReRoPE和ReRoPE的相对位置是分段线性的，这意味着朴素地实现的话，需要算两次Attention矩阵（得到两段不同的线性）然后将它们拼接起来，这样效率无疑明显降低了。

不过，好消息是当前主流的Attention加速手段如Flash Attention都是将Attention分块计算的，比如每128长度为一块，这样当序列足够长时，分段线性的块占比非常少（只有窗口边界附近），如下式所示，只有红绿混色的块才需要重复计算Attention，剩下同色的块都只需要计算一次，所以结合分块计算Attention的话，Leaky ReRoPE和ReRoPE所增加的计算成本时几乎可以忽略的。此前读者 @chu-tianxiang 在[评论区](#)也分享了一个基于Tri

ton的实现，大家有兴趣的可以参考一下。

$$\left(\begin{array}{cccc|ccc|ccc}
0 & & & & & & & & & & \\
1 & 0 & & & & & & & & & \\
2 & 1 & 0 & & & & & & & & \\
\ddots & 2 & 1 & 0 & & & & & & & \\
w-1 & \ddots & 2 & 1 & 0 & & & & & & \\
w & w-1 & \ddots & 2 & 1 & 0 & & & & & \\
w+\frac{1}{k} & w & \ddots & \ddots & 2 & 1 & 0 & & & & \\
w+\frac{2}{k} & w+\frac{1}{k} & \ddots & \ddots & \ddots & 2 & 1 & 0 & & & \\
\ddots & w+\frac{2}{k} & \ddots & \ddots & \ddots & \ddots & 2 & 1 & 0 & & \\
\ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \\
\ddots & \ddots & \ddots & w+\frac{2}{k} & w+\frac{1}{k} & w & w-1 & \ddots & 2 & 1 & 0 \\
w+\frac{L-1-w}{k} & \ddots & \ddots & \ddots & w+\frac{2}{k} & w+\frac{1}{k} & w & w-1 & \ddots & 2 & 1 & 0
\end{array} \right) \quad (3)$$

$e^{i(m-n)\theta_i}$ 实际上就单位圆上的点，当 $m-n$ 逐渐变大时，这个点就在单位圆上转圈（真·旋转）， θ_i 越大则转得越快，反之越慢。



假设训练长度为 L_{train} ，那么 $m-n \in [0, L_{train} - 1]$ ，接下来让我们充分发挥想象力：较大的 θ_i 意味着转速越快，周期越短，于是在 $m-n$ 从 0 到 $L_{train} - 1$ 期间，它已经被转了很多圈，也就是说圆上的每一个点几乎都被训练过，因此这些 θ_i 几乎不存在 OOD 问题；相反，对于较小的 θ_i ，当 $m-n$ 从 0 到 $L_{train} - 1$ 时它可能还没转完一圈，这种情况下被训练过的点顶多只是圆上的一条弧，如果测试时遇到更大的 L_{test} ，那么就超出了训练过的弧范围，从而有无法预估的表现，这时候就需要通过内插将它压缩到原本的弧内。说白了，位置标号 $m-n$ 是否 OOD 根本不重要，重要的是单位圆上的点是否被充分训练过，如果是，那么就可以不做改动（直接外推），否则就要想办法将它压缩到已经被充分训练过的那段弧上（位置内插）。

具体来说，对于 θ_i ，我们可以算出周期为 $T_i = 2\pi/\theta_i$ ，然后可以算出在训练过程中它所转的“圈数”为 $r_i = \frac{L_{train}}{T_i} = \frac{\theta_i L_{train}}{2\pi}$ ，我们可以设一个圈数的阈值 τ ，圈数超过 τ 的，就认为已经充分训练了，可以不加改动；圈数少于 1 的， θ_i 改为 $\frac{\theta_i L_{train}}{L_{test}}$ ，意味着要把超出弧范围的重新缩放到弧内；至于剩下的部分，就在两者之间线性插值过渡。用公式表达就是：

$$\theta_i^{new} = \left[\gamma_i + (1 - \gamma_i) \frac{L_{train}}{L_{test}} \right] \theta_i, \quad \gamma_i = \begin{cases} 1, & r_i > \tau \\ 0, & r_i < 1 \\ \frac{r_i - 1}{\tau - 1}, & \text{others} \end{cases} \quad (5)$$

这就是《YaRN: Efficient Context Window Extension of Large Language Models》一文所提出的免训练长度外推方案“YaRN”，在笔者的测试中，它的外推效果非常好，只是略逊于 Leaky ReRoPE 和 ReRoPE。但要注意的是，YaRN 只改变 θ_i 的值，不改变 Attention 和 RoPE 的形式，因此不会有额外的实现成本和推理成本，在满足这个条件之下（即可以完全代入已有的实现），YaRN 是笔者测试过的效果最佳的长度外推方法。

一些插曲

其实 YaRN 的故事还没完，不过感觉上一节已经很长了，还是另开一节比较好。YaRN 除了引入 θ_i 的改动外，还在 Attention 的 Logits 上多乘了一个 Scale 因子：

$$\lambda = \left(1 + 0.1 \log \frac{L_{test}}{L_{train}} \right)^2 \approx 1 + 0.2 \log \frac{L_{test}}{L_{train}} \quad (6)$$

关于这个 Scale 的推导，可能会让人有点啼笑皆非，答案是根本没有推导，作者说他也没能从理论上推导出来，纯粹是实验发现加了以上 Scale 后 PPL 更低，以上形式也是通过实验拟合出来的。

其实这个带对数的结果，很明显跟《从熵不变性看Attention的Scale操作》推导出来的 $\log n$ Scale非常相似，只不过后者跟具体位置有关，而前者在确定了 L_{test} 之后就是一个常数。考虑到当 n 比较大时， $\log n$ 函数变化比较缓慢，所以在一定范围内取为常数也无可厚非，因此，不难猜测YaRN的这个Scale因子跟熵不变性的 $\log n$ Scale应该是同源的。笔者也做过对比，将常数 λ 换成如下跟绝对位置 n 相关的因子，能起到相近的效果：

$$\lambda_n = \max \left(1, \frac{\log n}{\log L_{train}} \right) \quad (7)$$

注意到

$$\frac{\log L_{test}}{\log L_{train}} = 1 + \frac{1}{\log L_{train}} \log \left(\frac{L_{test}}{L_{train}} \right) \quad (8)$$

YaRN是基于LLAMA和LLAMA2做实验的，前者训练长度是2K，后者是4K，我们有 $\frac{1}{\log 2048} \approx 0.13$ ， $\frac{1}{\log 4096} \approx 0.12$ ，系数大致是式(6)的一半，差别不大，事实上这个系数的精确值可能不太重要，因为笔者也发现过式(7)更好的数据集，所以由此我们便算将式(6)近似地推导出来了。

相比YaRN本身，YaRN的作者Bowen Peng的故事也许更加称得上“引人入胜”，他早前所提出的NTK-RoPE是RoPE的第一个免训练的长度外推方案，本系列的两篇博客《Transformer升级之路：10、RoPE是一种 β 进制编码》和《Transformer升级之路：11、将 β 进制位置进行到底》都直接受启发于它。虽然从目前来看，NTK-RoPE的效果不见得多好（相比YaRN、ReRoPE等），但它首次显示了免训练长度外推的可能性，具有里程碑式的意义，甚至可以说，后续的所有长度外推相关研究，都直接或者间接得益于NTK-RoPE打开了大家的想象力。

NTK-RoPE的思路很简单，仅改一下RoPE的base就行，即原本是 $\theta_i = 10000^{-2i/d}$ ，现在改为 $\theta_i = (10000\kappa)^{-2i/d}$ 。 κ 怎么选取呢？当时Bowen Peng基于自己对NTK（Neural Tangent Kernel）相关结果的经验，判断高频（ $i \rightarrow 0$ ）是学习相对距离的，所以不用改变，低频（ $i \rightarrow d/2 - 1$ ）是学习绝对距离的，因此要进行内插，总结起来就是“高频外推、低频内插”，于是他通过令 $i = d/2 - 1$ 时的Scale正好等于内插Scale $\frac{L_{train}}{L_{test}}$ ，得出方程

$$(10000\kappa)^{-2i/d}|_{i=d/2-1} = \frac{L_{train}}{L_{test}} 10000^{-2i/d}|_{i=d/2-1} \quad (9)$$

解得

$$\kappa = \left(\frac{L_{test}}{L_{train}} \right)^{d/(d-2)} \quad (10)$$

就这么个简单且高明的推导，打开了免训练长度外推的“潘多拉魔盒”。

从YaRN的视角看，并非只有 $i = d/2 - 1$ 时的 θ_i 才转得不足一周，所以NTK-RoPE只让最后一个 $i = d/2 - 1$ 做完整的内插，是不够充分的，事实上确实也是如此，设置如式(10)的 κ ，实际上只能让模型外推到 $L_{test}/2$ 左右的长度而不发生PPL爆炸，再长PPL就明显上升了。也就是因为有这个问题，作者才进一步提出了后来的升级版方案YaRN。

不过，尽管NTK-RoPE效果上不如YaRN，但对于前面提到的第二种有资源去做长文本微调的读者，可能会更喜欢NTK-RoPE，因为他们只是为了得到一个更好的初始化模型，反正都是要微调，NTK-RoPE与YaRN的初始效果差异他们并不会太在意，相比之下他们更乐意选择实现更简单的NTK-RoPE了，比如CodeLLAMA就是在LLAMA2的基础上将base改为 10^6 然后继续训练的。此外，Meta在其论文《Effective Long-Context Scaling o

《f Foundation Models》中，将NTK-RoPE改称为RoPE-ABF (Adjusted Base Frequency)，相比神秘的NTK，ABF的名称能更直观体现出它的含义。

拒绝交税

不知道大家留意到没有，上面提到的免训练长度外推方法，都无法使得模型在训练长度 L_{train} 内的效果保持不变。具体来说，设原本模型为 $f(x)$ ，做了外推改动后的模型是 $f^+(x)$ ，当 x 的长度不超过 L_{train} 时，无法保证 $f(x) \equiv f^+(x)$ 。由于 $f(x)$ 就是在 L_{train} 内训练的，因此可以合理地认为 $f(x)$ 对于长度不超过 L_{train} 的样本效果是最优的，于是 $f^+(x) \neq f(x)$ 意味着长度外推虽然使得更长的样本效果变好了，但原本 L_{train} 内的效果却变差了。我们可以形象将这部分损失称为“外推税”。

早在NTK-RoPE刚提出那会，开源社区就意识到了“外推税”的问题，并提出了对应的解决办法——随着训练长度的变化动态地调整各个外推方法的Scale因子，这就是Dynamic Scaling，最早提出自Reddit的帖子《Dynamically Scaled RoPE further increases performance of long context LLaMA with zero fine-tuning》。以YaRN为例，里边跟长度相关的缩放因子是 $s = \frac{L_{test}}{L_{train}}$ ，Dynamic Scaling将它换成动态的 $s(pos) = \frac{\max(L_{train}, pos+1)}{L_{train}}$ ，其中 pos 是当前Token的位置编号（从零开始算）。这个改动意味着Dynamic Scaling试图给每个位置都找到最小的、理论上对模型效果影响也最小的Scale因子（或者等价地，每个位置都配不一样的 $\theta_i(pos)$ ），从而达到拒绝交税的效果。

不过，要想真正实现每个位置都有不一样的 $\theta_i(pos)$ 是很困难的。跟Leaky ReRoPE和ReRoPE需要重复计算Attention的原因一样，因为RoPE是通过绝对位置的方式实现相对位置，这意味着单次计算只能实现一个固定的 θ_i ，不同位置要想实现不同的 θ_i ，那么KV Cache中的K只能存Apply RoPE之前的，并且不同位置得分别计算多次，这就变成了类似RNN的递归过程。我们知道LLM回复一轮对话，可以分为prefill和generation两个阶段，prefill指的是对输入部分的计算，generation就是token by token的生成阶段。很明显prefill阶段原本是可并行的，如果也改为像generation那样的递归，那么在输入很长（比如输入一篇论文）时，无疑会明显拖慢计算速度，因此变得不大实际。

于是，一个折中的方法是“局部静态”：prefill阶段的输入有多少个Tokens我们是计算得到的，然后generation阶段我们也会设置一个max_gen_tokens，我们将这两个数字相加，作为当前这一轮对话的 L_{test} 去计算对应的 θ_i ；完成这一轮对话后，下一轮对话我们再用同样的方式更新 L_{test} 和 θ_i 。如此一来，就不用给模型引入过于复杂或者牺牲效率的实现了，算是一个比较实用的方案，尤其是当输入很长时，max_gen_tokens远小于prefill的tokens，在单轮对话中的Scale本来就近似为常数。

Dynamic Scaling的思想，可以说被《CLEX: Continuous Length Extrapolation for Large Language Models》提出的CLEX发挥到了极致：CLEX同样要给每个位置都配不一样的 $\theta_i(pos)$ ，它将 $\theta_i(pos)$ 假设为关于 pos 的连续函数，并且用一个神经ODE来建模，通过微调来拟合这个ODE的参数，最终取得了比YaRN更好的结果，并且实验结果显示不断地Dynamic Scaling下去，可以得到近乎无限的长度外推能力。

另起炉灶

除了Dynamic Scaling外，“拒绝交税”的另一个思路是“另起炉灶”，通过重新设计预训练时所用的模型架构，使得它具备训练完成后就可以不做任何修改实现长度外推的潜力，在这个系列的文章中，笔者有两篇相关的探讨，分别是在《Transformer升级之路：9、一种全局长度外推的新思路》所提到HWFA (Hybird Window-Full Attention)，以及在《Transformer升级之路：15、Key归一化助力长度外推》所验证的Key Norm。

其中，HWFA是将模型的前 $L - 1$ 层Attention都换成窗口很小的RoPE + Window Attention，而最后一层Attention则换成NoPE + Full Attention，这样修改之后训练出来的模型，不加改动就有一定的长度外推效果。包含类似思想的还有《[Focused Transformer: Contrastive Training for Context Scaling](#)》，不过这篇不是做长度外推的，而是想要通过简单微调来拓展LLM的Context长度。HWFA的问题是在训练效果上会逊色于标准的Attention模型，为此笔者后来在《[Transformer升级之路：14、当HWFA遇见ReRoPE](#)》提出了改进版的HWFA2（即HWFA + ReRoPE）。

相比HWFA，HWFA2的Window Attention使用了更大的Window Size，并且恢复了Full Attention的RoPE，同时允许多于一层的Full Attention穿插在Window Attention之间（而不只是一层放在最后），这样的修改可以追平与标准Attention在训练效果上的差距（甚至偶尔会更优），但缺点是不能够不加改动就实现长度外推了（需要将RoPE换ReRoPE），也算是有得有失。当然，我们也可以不看外推效果，纯粹将HWFA2看成是一种不损失效果而且明显降低模型复杂度的加速方案。顺便说，上个月Arxiv的一篇论文《[Zebra: Extending Context Window with Layerwise Grouped Local-Global Attention](#)》提出了名为Zebra的方法，它跟HWFA2都是若干层Full Attention穿插在Window Attention中的组合方式。

至于Key Norm，则是源于将Attention的Key做了L2归一化后模型长度外推能力居然明显变好了的“意外发现”，对它的进一步思考加深了笔者对长度外推的理解。对于标准的基于Q、K内积的Attention，我们可以将它表示为

$$s(n|m) = \mathbf{q}_m \cdot \mathbf{k}_n = \|\mathbf{q}_m\| \|\mathbf{k}_n\| \cos(\mathbf{q}_m, \mathbf{k}_n), \quad p(j|i) = \frac{\exp\left(\frac{s(n|m)}{\sqrt{d}}\right)}{\sum_{j=1}^i \exp\left(\frac{s(n|m)}{\sqrt{d}}\right)} \quad (11)$$

很明显，要想提高 n 对某个 m 的相对注意力，模型有两个选择：增大 $\|\mathbf{k}_n\|$ ，或者增大 $\cos(\mathbf{q}_m, \mathbf{k}_n)$ 。由于维度灾难的原因，增大 $\|\mathbf{k}_n\|$ 会比增大 $\cos(\mathbf{q}_m, \mathbf{k}_n)$ 更加容易，所以如果有可能的话，模型会尽可能选择增大 $\|\mathbf{k}_n\|$ ，而 $\|\mathbf{k}_n\|$ 跟 i 无关，描述的是绝对重要性，这可能是Scissorhands所描述的注意力分布特点的成因之一。另一方面，模型倾向于选择增大 $\|\mathbf{k}_n\|$ ，那么意味着对 $\cos(\mathbf{q}_m, \mathbf{k}_n)$ 的训练可能不充分，这大概是Attention无法长度外推的更本质的原因。

由此，Key Norm能够改善长度外推能力的原因就豁然开朗了。Key Norm将所有的 $\|\mathbf{k}_n\|$ 都归一化为1，模型便没有了“增大 $\|\mathbf{k}_n\|$ ”这个选择，于是只能一心调整 $\cos(\mathbf{q}_m, \mathbf{k}_n)$ ，使得 $\cos(\mathbf{q}_m, \mathbf{k}_n)$ 的训练更加充分。同时，笔者也做过对比实验，Key Norm只有在跟RoPE配合才能体现出长度外推能力，Key Norm + NoPE或者单纯的NoPE都没有长度外推效果，这大概也是RoPE本身的旋转作用，丰富了 $\mathbf{q}_m, \mathbf{k}_n$ 夹角的多样性（像是数据扩增），从而让 $\cos(\mathbf{q}_m, \mathbf{k}_n)$ 的训练更加充分了。

还有一篇名为《[CoCA: Fusing position embedding with Collinear Constrained Attention for fine-tuning free context window extending](#)》的论文从另外的角度提出了一个解决办法：它通过修改注意力的实现方式，使得每一组 $\mathbf{q}_m^{(i)}, \mathbf{k}_m^{(i)}$ 都有 $\cos(\mathbf{q}_m^{(i)}, \mathbf{k}_m^{(i)}) = 1$ ，这里的分组 i 就是前述的RoPE对 $\mathbf{q}, \mathbf{k}$ 分量的两两分组，这样设计会使得比较大的 $\cos(\mathbf{q}_m, \mathbf{q}_n)$ 尽量都被训练过（ $\cos$ 最大也就是1），训练不充分的也只是小的部分（这部分Softmax后的概率会比较小，不会明显干扰注意力分布），从而获得一定的长度外推的能力。不过CoCA对注意力的修改，有降低每个注意力头的能力上限的风险，即相同参数下，它可能只有 $\text{head_size}/2$ 的标准注意力头的拟合能力。

其他思路

写到这里，对长度外推的介绍也已经尾声了。尽管已经写了相当长的篇幅，但依然很难对所有长度外推的工作都做详细介绍。下面零散地列举一些能想起来的其他相关工作。

一开始，我们认为Attention不能长度外推是因为预测时位置的“越界”，对此一个朴素的解决方法是对训练阶段的位置编码进行扰动，即类似数据扩增的方式，以求让模型提前适应预测所用的位置编码，本系列的《Transformer升级之路：8、长度外推性与位置鲁棒性》和《Transformer升级之路：13、逆用Leaky ReRoPE》都可归入这类，此外还包括几个月前的《PoSE: Efficient Context Window Extension of LLMs via Positional Skip-wise Training》。这类方法在笔者的实验中不算太稳定，而且带来了额外的复杂度或者随机性，很难保证不会影响模型原本的Scaling Law。

有些读者曾提出过疑问：在YaRN的分析中，要做插值的是低频部分，那如果干脆直接去掉低频部分会怎么样？或者类似地，将base调小，让高频部分的比例增加？笔者缺失在预训练中尝试过调小RoPE的base，结果是最终效果更差了，并且也没表现出长度外推能力。不过《Scaling Laws of RoPE-based Extrapolation》（知乎有中文版《RoPE外推的缩放法则——尝试外推RoPE至1M上下文》）尝试过另一种方案，是在微调阶段才调小Base，配合短文本的微调后能体现出长文本的外推能力。

但从笔者的角度看，调小Base甚至去掉低频的做法并不科学，即便它可能在某些情况下有长度外推效果，但可能损失掉模型本身的能力。就像NTK-RoPE、YaRN的作者Bowen Peng曾经的观点，高频学习到的是局部的相对距离，低频学习到的是远程的绝对距离，两者都很重要，它们之间更像是一种层次的关系；用《Transformer升级之路：10、RoPE是一种 β 进制编码》的进制类别来看，低频对应的就是高位，如果只保留低位而去掉高位，那么结果就相当于求模（余数），无法准确表达出位置信息来；更何况，高频和低频其实是相对的，一个频率对10K长度的文本来说是低频，但对于100K长度的文本来说可能就是高频了。

近来还有一个有意思的论文《Fortify the Shortest Stave in Attention: Enhancing Context Awareness of Large Language Models for Effective Tool Use》，它发现同一个模型改不同的base然后将输出取平均，能增强模型的整体性能，这表明不同大小的base各有所长，不能单纯为了外推而去调小它。

总的来说，长度外推技术虽然有了长足的进展，但依然还是一件很神秘的事情。比如，推理阶段将RoPE换成ReRoPE，就能体现出一定的长度外推效果，那么预训练阶段就换成ReRoPE，是不是外推效果就更好？恰恰相反，笔者做过训练阶段就切换为ReRoPE的实验，结果训练出来的模型没有一丁点长度外推能力。这大体也跟Key Norm那里的分析有关，训练阶段就换ReRoPE降低了 q_n, k_m 夹角的多样性，反而让 $\cos(q_n, k_m)$ 的训练没那么充分，从而降低了长度外推能力。还有很多长度外推技术可能跟架构绑定的，早年的一些据说有长度外推能力的位置编码，包括ALIBI、KERPLE、XPOS等，都是在Multi-Head Attention + Pre Norm上做的实验，而笔者在Single Head的GAU + Post Norm上，从未测出它们有长度外推能力，这表明关于长度外推的分析，很可能还缺少了架构相关的那一环。

文章小结

在这篇文章中，笔者结合自己的学习经历，梳理了过去一年来关于长度外推的相关进展，主要简明地介绍了相关方法的特点和背后的思想，并试图将它们串联起来，希望本文能帮助大家更深入、更系统地了解长度外推这个课题。如果有什么错漏之处，也请读者提醒和指正。

转载到请包括本文地址：<https://spaces.ac.cn/archives/9948>

更详细的转载事宜请参考：《科学空间FAQ》

如果您需要引用本文，请参考：

苏剑林. (Jan. 26, 2024). 《Transformer升级之路：16、“复盘”长度外推技术 》 [Blog post]. Retrieved from <https://spaces.ac.cn/archives/9948>

```
@online{kexuefm-9948,
  title={Transformer升级之路：16、“复盘”长度外推技术},
  author={苏剑林},
  year={2024},
  month={Jan},
  url={\url{https://spaces.ac.cn/archives/9948}},
}
```